CI/CD for Vue.js Techcnology App

Course Name: Devops Foundation

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Pranjal Mahwar	EN22CS301729
2.	Prateeksha Shukla	EN22CS301740
3.	Parthiv Prajapat	EN22EL301039
4.	Raj Patel	EN22CS301781
5.	Rishabh Dev Rajput	EN22CS17803

Group Name: 06D7

Project Number: D0-29

Industry Mentor Name: Aashruti Shah

University Mentor Name: Shyam Patel

Academic Year:2025-2026

Problem Statement & Objectives

1. Problem Statement
2. Project Objectives
3. Scope of the Project

Proposed Solution

1. Key features *(Just mention key features here no need to go into details)*
2. Overall Architecture / Workflow
3. Tools & Technologies Used (*If applicable)*

Results & Output

Add the below details here:

1. *Screenshots / outputs*
2. *Reports / dashboards / models*
3. *Key outcomes*

Conclusion

Mention a brief conclusion about your project summing up everything you have worked on and the key learning.

Future Scope & Enhancements

Problem Statement & Objectives

1. Problem Statement

In modern software development, delivering applications quickly and reliably is a major challenge. Traditional deployment methods are manual, time-consuming, and prone to human errors, which can lead to inconsistent builds and downtime.

For a Vue.js application, there is a need to automate the entire software delivery lifecycle—from code integration to deployment—using a robust CI/CD pipeline.

This project aims to design and implement a **CI/CD pipeline using AWS EC2 instances** along with tools like Git, Docker, and GitHub Actions/Jenkins to:

- Automate build and testing processes
- Containerize the application using Docker
- Deploy the application efficiently on AWS EC2
- Ensure high availability and reliability

2. Project Objectives

The main objectives of this project are:

2.1 Automation of CI/CD Pipeline

- Automate code integration, build, and deployment processes
- Trigger pipeline automatically on every code push

2.2 Continuous Integration (CI)

- Fetch code from GitHub repository
- Install dependencies and build Vue.js application
- Run automated tests (if applicable)

2.3 Continuous Deployment (CD)

- Build Docker image for the application
- Push Docker image to DockerHub
- Deploy container on AWS EC2 instance

2.4 Use of AWS EC2

- Host the application on EC2 virtual server
- Configure server environment (Docker, Node.js, etc.)
- Enable remote deployment

2.5 Containerization

- Use Docker to ensure consistent environments
- Simplify deployment and scalability

2.6 Reliability & Scalability

- Ensure fast and error-free deployments
- Make system scalable for future enhancements

3. Scope of the Project (Using AWS EC2 Instances)

This project focuses on building a **complete DevOps pipeline** for a Vue.js application with deployment on AWS EC2.

3.1 Included in Scope

- Development of a Vue.js application
- Version control using GitHub
- CI/CD pipeline using GitHub Actions or Jenkins
- Docker container creation and management
- Deployment on AWS EC2 instance
- Basic monitoring and logging

3.2 AWS EC2 Usage

- Launch and configure EC2 instance (Ubuntu/Linux)
- Install Docker and required dependencies
- Host and run the application container
- Access application via public IP

3.3 Deployment Workflow

1. Developer pushes code to GitHub
2. CI/CD pipeline triggers automatically
3. Application is built and tested
4. Docker image is created and pushed to DockerHub
5. EC2 instance pulls latest image
6. Container is deployed and application goes live

3.4 Limitations

- No advanced auto-scaling (like AWS ECS/EKS)
- Limited to single EC2 instance deployment
- Basic security configurations

3.5 Future Enhancements

- Integration with Kubernetes (EKS)
- Load balancing using AWS ELB
- Monitoring using CloudWatch
- Auto-scaling setup

Proposed Solution

1. Key Features

The proposed system incorporates several important features to ensure an efficient and reliable CI/CD pipeline:

- **Automated CI/CD Pipeline:**
  The entire process from code commit to deployment is automated, reducing manual effort and errors.
- **Continuous Integration:**
  Every code push triggers automatic build and testing, ensuring early detection of issues.
-

- **Continuous Deployment:**
  After successful build and testing, the application is automatically deployed to the AWS EC2 server.
- **Docker-Based Containerization:**
  The application is packaged into Docker containers to ensure consistency across environments.
- **Cloud Deployment on AWS EC2:**
  The application is hosted on AWS EC2, providing scalability and remote accessibility.
- **Version Control Integration:**
  GitHub is used to manage source code, track changes, and collaborate efficiently.
- **Automated Image Management:**
  Docker images are built and pushed to Docker Hub for versioning and reuse.
- **Fast and Reliable Releases:**
  The pipeline ensures quick deployments with minimal downtime.
- **Environment Consistency:**
  Docker ensures the application runs the same way in development, testing, and production.
- **Extensibility:**
  The system can be easily extended to include Kubernetes, monitoring tools, and auto-scaling.

2. Overall Architecture / Workflow

The proposed solution follows a structured DevOps pipeline that automates the entire software delivery lifecycle.

Step-by-Step Workflow:

Step 1: Code Development

- Developers write the Vue.js application code locally.
- Code is pushed to a GitHub repository.

Step 2: Trigger CI/CD Pipeline

- GitHub Actions or Jenkins automatically triggers the pipeline when:
  - Code is pushed to the main branch
  - A pull request is merged

Step 3: Build Process (CI Phase)

- The pipeline performs:
  - Dependency installation (npm install)
  - Application build (npm run build)
- Ensures the application compiles successfully.

Step 4: Testing (Optional but Recommended)

- Automated tests are executed.
- Ensures code quality and functionality before deployment.

Step 5: Docker Image Creation

- A Docker image is built using a Dockerfile.
- The image contains the Vue.js application and required runtime environment.

Step 6: Push to Docker Hub

- The Docker image is tagged and pushed to Docker Hub.
- Acts as a centralized storage for application images.

Step 7: Deployment to AWS EC2 (CD Phase)

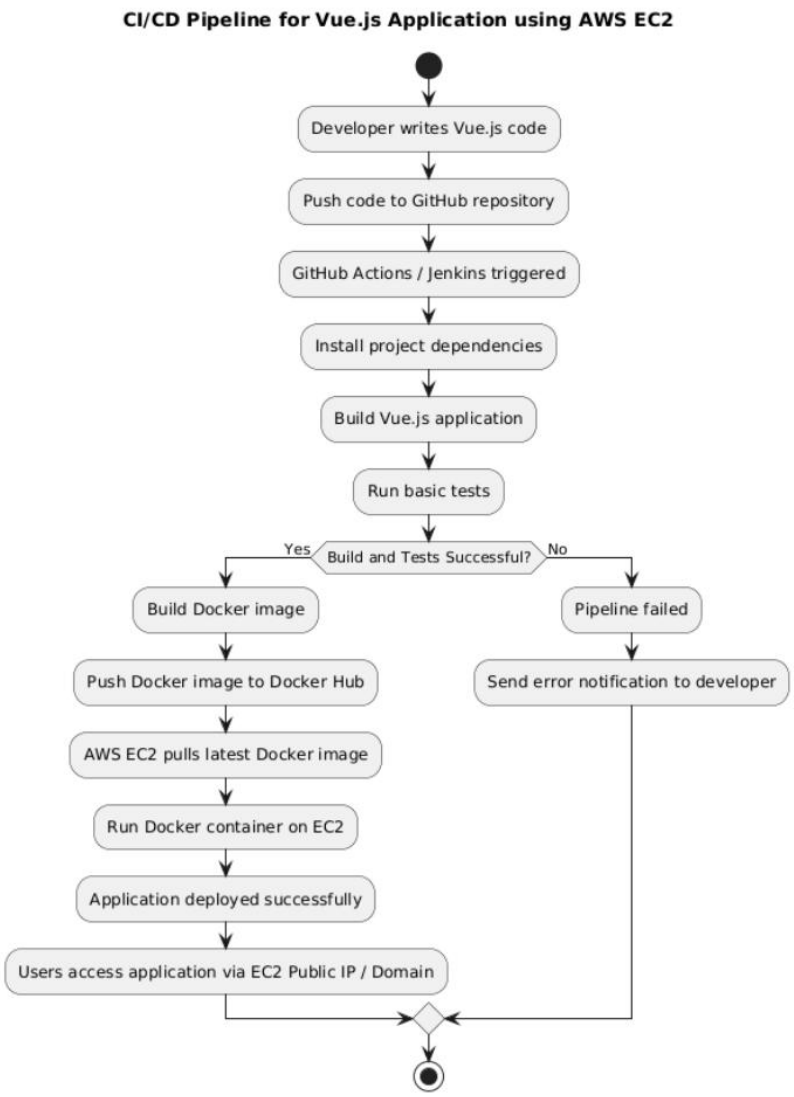
- AWS EC2 instance is pre-configured with Docker.
- The EC2 server pulls the latest Docker image from Docker Hub.

Step 8: Container Execution

- Docker container is run on EC2:
  - Application becomes live on a specific port (e.g., 80 or 8080)
- Users can access the application via EC2 public IP.

Step 9: Continuous Updates

- Every new code push repeats the pipeline.
- Ensures the application is always updated with the latest version.



3. Tools & Technologies Used

This project uses a combination of modern DevOps tools and cloud technologies:

Frontend Technology

- **Vue.js**
  Used to develop the user interface of the application. It is lightweight, fast, and component-based.

Version Control System

- **Git**
  Tracks changes in the source code.
- **GitHub**
  Hosts the repository and integrates with CI/CD tools.

CI/CD Tools

- **GitHub Actions / Jenkins**
  - Automates build, test, and deployment processes
  - Executes workflows based on triggers like code push

Containerization

- **Docker**
  - Packages the application into containers
  - Ensures consistent execution across environments

Container Registry

- **Docker Hub**
  - Stores Docker images
  - Allows EC2 instance to pull latest application image

Cloud Platform

- **Amazon Web Services (AWS)**

AWS EC2 (Elastic Compute Cloud)

- Provides virtual servers to host the application
- Allows remote access and deployment
- Scalable and widely used in industry

Operating System

- **Ubuntu/Linux**
    - Installed on EC2 instance
    - Used for running Docker and hosting application

Web Server / Runtime

- **Nginx / Node.js**
    - Serves the Vue.js application
    - Handles incoming requests

Deployment & Access

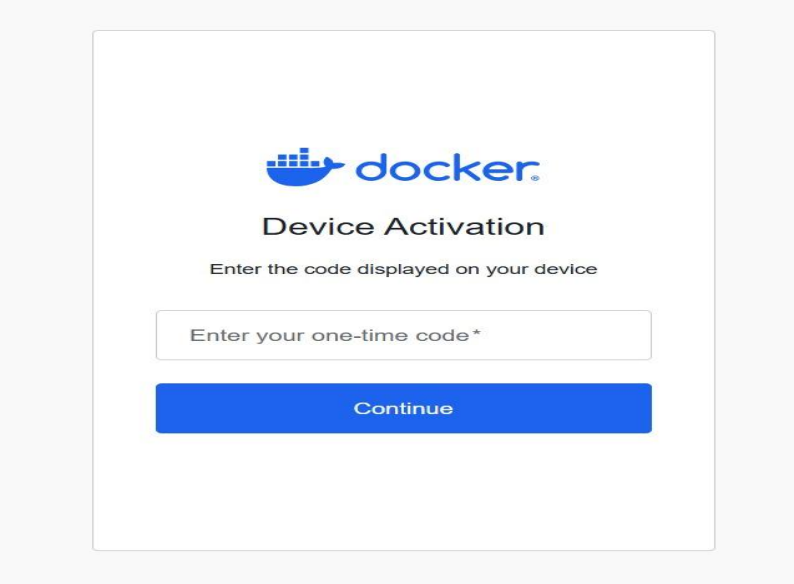
- **SSH (Secure Shell)**
    - Used to connect to EC2 instance remotely
    - Execute deployment commands

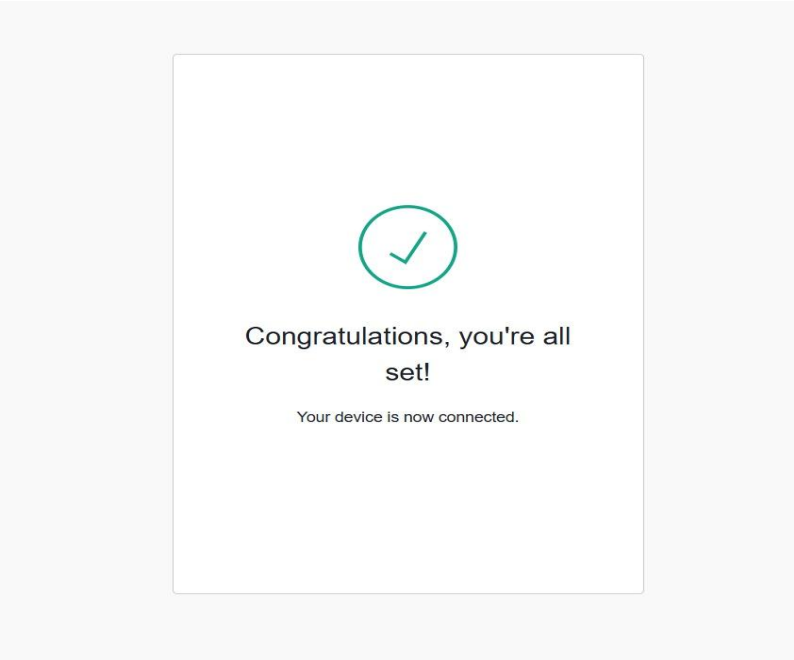
Future Technologies (Optional Enhancements)

- **Kubernetes (K8s):** For container orchestration
- **AWS CloudWatch:** For monitoring and logging
- **AWS Load Balancer:** For handling traffic
- **Auto Scaling:** For handling increased load

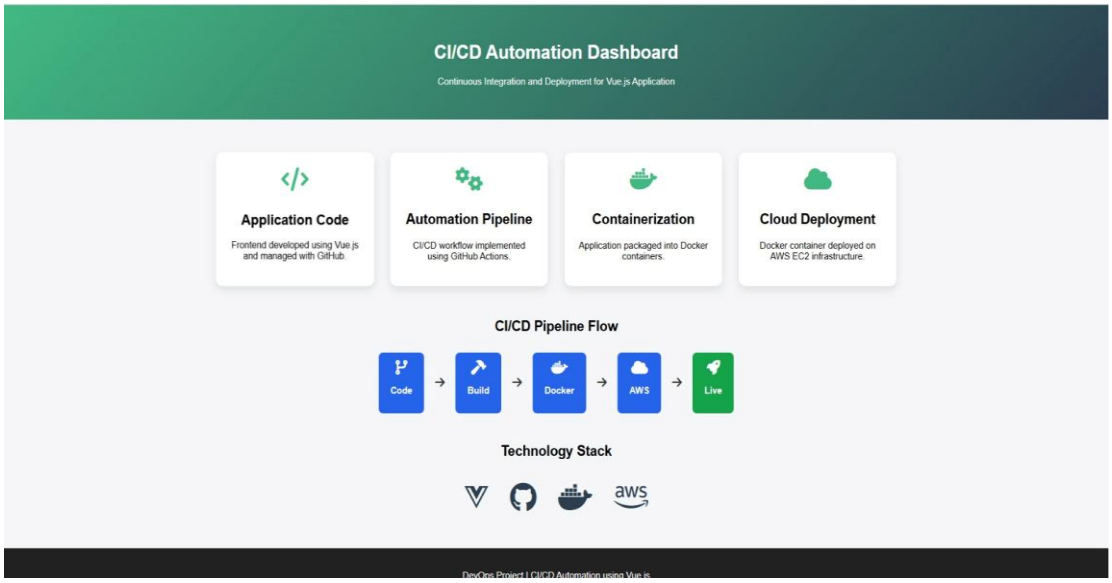
Results & Output

Docker Activation

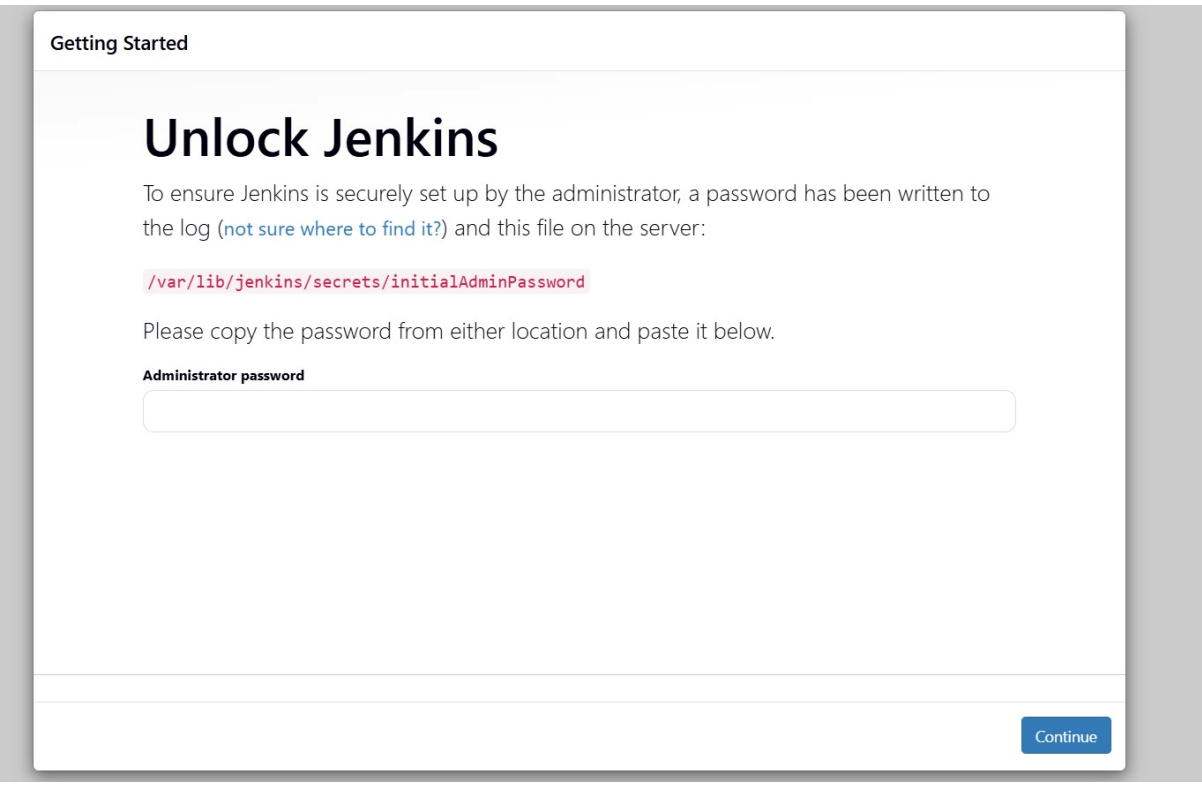




User Interface



Jenkins Implementation



Conclusion

In this project, we successfully designed and implemented a complete **CI/CD pipeline for a Vue.js application** using modern DevOps tools and cloud infrastructure. The system integrates **GitHub for version control, GitHub Actions/Jenkins for automation, Docker for containerization, and AWS EC2 for deployment**, enabling a fully automated software delivery process.

The pipeline ensures that every code change goes through a standardized process of **build, test, packaging, and deployment**, which significantly reduces manual intervention and minimizes errors. By using Docker, we achieved consistency across different environments, while AWS EC2 provided a reliable and scalable platform to host the application.

Throughout this project, we gained valuable hands-on experience in:

- Setting up and managing CI/CD pipelines
- Working with Docker for containerization
- Deploying applications on AWS EC2 instances
- Automating workflows using GitHub Actions/Jenkins
- Understanding real-world DevOps practices and workflows

This project demonstrates how DevOps practices can improve **software quality, deployment speed, and system reliability**. It also lays a strong foundation for future enhancements such as Kubernetes-based orchestration, auto-scaling, and advanced monitoring.

Overall, the project helped in developing both **technical skills and practical understanding of end-to-end DevOps implementation**, making it highly relevant for real-world industry applications.

Future Scope & Enhancements

The current project establishes a strong foundation for implementing a CI/CD pipeline using AWS EC2. However, it can be further enhanced to meet enterprise-level requirements and improve scalability, security, and performance.

1. Container Orchestration with Kubernetes

- Integrate **Kubernetes (K8s)** for managing multiple containers
- Enable auto-scaling and self-healing of applications
- Use AWS EKS (Elastic Kubernetes Service) for managed orchestration

2. Auto Scaling and Load Balancing

- Implement **AWS Auto Scaling** to handle varying user traffic
- Use **Elastic Load Balancer (ELB)** to distribute traffic across instances
- Improve application availability and performance

3. Advanced Monitoring and Logging

- Integrate **AWS CloudWatch** for real-time monitoring
- Use tools like **Prometheus and Grafana** for metrics visualization
- Enable centralized logging for better debugging and analysis

4. Enhanced Security Practices

- Implement **IAM roles and policies** for secure access control
- Use **HTTPS with SSL/TLS certificates** for secure communication
- Integrate **DevSecOps practices** such as vulnerability scanning in CI/CD pipeline

5. Infrastructure as Code (IaC)

- Use tools like **Terraform or AWS CloudFormation**
- Automate infrastructure provisioning and configuration
- Ensure consistency and repeatability of environments

6. Multi-Environment Deployment

- Create separate environments like:
  - Development
  - Staging
  - Production
- Enable safer testing before production deployment

7. Blue-Green / Canary Deployment

- Implement advanced deployment strategies:
  - **Blue-Green Deployment** (zero downtime release)
  - **Canary Deployment** (gradual rollout)
- Reduce risks during application updates

8. CI/CD Pipeline Enhancements

- Add **automated testing (unit, integration, UI testing)**
- Include **code quality tools** like SonarQube
- Improve pipeline efficiency and reliability

9. Domain and DNS Configuration

- Replace EC2 public IP with a **custom domain name**
- Use **AWS Route 53** for DNS management
- Improve user accessibility and branding

10. Microservices Architecture (Advanced)

- Convert monolithic Vue.js backend integration into **microservices**
- Deploy services independently using containers
- Improve scalability and maintainability

Conclusion of Future Scope

These enhancements will transform the project from a basic CI/CD implementation into a **fully scalable, secure, and enterprise-grade DevOps system**. Adopting these improvements will align the project with industry standards and prepare it for real-world production environments.